CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

A MINI PROJECT REPORT ON

A CLOUD-LITE BIG DATA PIPELINE FOR F1 PERFORMANCE ANALYSIS AND VISUALIZATION

Submitted in partial fulfillment of the requirements for the degree of

MASTER OF TECHNOLOGY
in
DATA SCIENCE

By

S OLIVE KERAN
2567218

Under the Guidance of

Dr. DIVYA VETRIVEERAN
Assistant Professor

Dr. REKHA V
Associate Professor

Department of AIML & Data Science
School of Engineering and Technology
CHRIST (Deemed to be University)
Kumbalagodu, Bangalore - 560 074

November 2025

Department of AIML & Data Science
School of Engineering and Technology
CHRIST (Deemed to be University)

BONAFIDE CERTIFICATE

This is to certify that S OLIVE KERAN, 2567218, has successfully completed the Mini Project entitled **"A Cloud-Lite Big Data Pipeline for F1 Performance Analysis and Visualization"** in partial fulfillment for the award of Master of Technology in Data Science during the academic year **2025-2026**.

Supervisor	**Mentor**	**Head of the Department**
Dr. Divya Vetriveeran	Dr. Rekha V	
Assistant Professor	Associate Professor	

Examiner 1:

(Name & Signature)

Examiner 2:

(Name & Signature)

INDEX

	CONTENT		PAGE NO.
	Abstract		i
	List of Symbols and Abbreviations		ii
	List of Figures		iii
1	**Introduction**		**1**
	1.1	Problem statement	1
	1.2	Motivation	1
	1.3	Research objectives	2
	1.4	Chapter wise summary	2
2	**Background Study**		**4**
	2.1.	Literature review	4
	2.2.	Data sources	5
	2.3	Articles	6
	2.4	Inferences	8
3	**Proposed Methodology**		**9**
	3.1	System architecture	9
	3.2	Methodology	11
	3.3	Module description	12
	3.4	Implementation	13
4	**Results and Discussion**		**15**
5	**Conclusion and Future Scope**		**23**
	5.1	Conclusion	23
	5.2	Future Scope	24
6	**Appendixes**		**25**
		Pseudocode	25
7	**References**		**30**

ABSTRACT

Formula 1, in the modern era of data-driven motorsport, involves large amounts of telemetry and timing data at a granular level, depicting the nuances of car performance and driver behavior. This project provides a big data framework that is scalable, incorporating open-source and cloud-lite technologies that gather, process, and analyze Formula 1 telemetry and lap data. The architecture proposed herein encompasses the FastF1 API as a primary data source and uses PySpark and DuckDB for efficient data transformation, querying, and analytics computation. Google Drive offers a light persistent storage solution, while Streamlit enables interactivity in terms of visualization, deploying on Hugging Face Spaces. The framework emulates real-time data velocity based on Python-based streaming and also allows strict data validation for veracity. The result shows advanced analytics, such as lap-time trends, driver performance comparison, and strategy insights that can be derived from high-volume racing telemetry. This work underlines the feasibility of implementing a cost-effective, scalable, and reproducible big data pipeline for Formula 1 analytics without dependence on commercial cloud platforms.

LIST OF SYMBOLS AND ABBREVIATIONS

CSV : Comma Separated Version

DRS : Drag Reduction System

F1 : Formula 1

ML : Machine Learning

KPI : Key Performance Indicators

LIST OF FIGURES

No.	Figure Name	Page No.
2.1	F1 telemetry analysis by the simulator F1 2021	5
2.2	Analytics dashboard based on telemetry data	6
2.3	Car sensor data	7
2.4	AWS ML Data Factory with F1 Data	8
3.1	Architecture of the proposed methodology	9
4.1	Data processed in batches	15
4.2	Anomaly detection in the dataset	16
4.3	Data cleaning in batches	16
4.4	Merging the cleaned datasets	17
4.5	Telemetry data summary	17
4.6	Counting number of laps for each driver	18
4.7	Performance based on tyre compound and stint length	18
4.8	Throttle Vs Break pattern	19
4.9	Average speed per 500m segment	20
4.10	Telemetry over session time	21

CHAPTER 1

INTRODUCTION

Formula 1 is one of the most technologically advanced and data-intensive sports globally. Each car has hundreds of sensors onboard that transmit telemetry data about speed, throttle, brake pressure, gear shifts, and more, to the team in pit lane, sending back several gigabytes of data within one race weekend. The analysis of this data becomes important for a clear understanding of vehicle dynamics, optimization of racing strategy, and enhancement of driver performance. Traditional methods of data analysis are not apt to manage or process such high-volume, high-velocity data. The project proposes a big data framework that can efficiently process, transform, and analyze Formula 1 telemetry and lap data in a scalable way with open-source tools. Integration of PySpark, DuckDB, and Streamlit within the Google Colab-Drive environment will show how big data principles such as volume, velocity, variety, and veracity can be implemented cost-effectively in an accessible way for motorsport analytics.

1.1 PROBLEM STATEMENT

The key problem this research aims to solve is the absence of publicly available, easily scalable, and affordable systems for processing and analyzing massive F1 telemetry and lap data. The few F1 analytics frameworks currently available either use commercial cloud infrastructure or are limited to small-scale, static datasets. Thus, researchers and enthusiasts interested in understanding detailed telemetry insights or attempting dynamic real-time analysis are significantly limited. The core of the problem the study tackles is the need for a free, open-source, modular system that would simulate big data architecture for Formula 1 analysis.

1.2 MOTIVATION

Motorsport analytics has traditionally been reserved for teams with access to proprietary tools and high-end computational resources. However, the availability of the FastF1 API and advancements in open-source big data technologies provide an opportunity to democratize F1 data analysis. The motivation behind this project lies in bridging the gap between academic

research and practical motorsport analytics—making high-volume data handling, transformation, and visualization achievable using free and accessible tools. Furthermore, the project aims to inspire data enthusiasts to explore real-world big data applications through the lens of Formula 1, a domain where milliseconds and micro-decisions define outcomes.

1.3 RESEARCH OBJECTIVES

The objectives of this research are:

- To design a scalable big data architecture for collecting, processing, and analysing Formula 1 telemetry and lap timing data.
- Ingestion, cleaning, and transformation pipelines will be implemented using tools such as PySpark and DuckDB, with a focus on schema validation and quality assurance.
- Perform analytical processing and visualisation to get driver and team performance insights at detailed levels.
- To simulate key big-data concepts like streaming ingestion, query optimization, and parallel processing in an accessible, cloud-lite environment.
- To deploy an interactive analytical dashboard as a Streamlit app, or in Hugging Face Spaces, for the end-user to explore the results.

1.4 CHAPTER WISE SUMMARY

CHAPTER 1 : INTRODUCTION

This chapter introduces the domain of the project, outlines the problem with the existing systems, the motivation and objectives of the research. It mentions the tools and techniques addressed in the following sections.

CHAPTER 2 : BACKGROUND STUDY

Surveys prior work in motorsport telemetry analytics, big data processing frameworks and open-source toolchains. It summarizes the existing literature and articles from across multiple reputed sources.

CHAPTER 3 : PROPOSED METHODOLOGY

Details the blueprint of the proposed system design, data flow architecture of ingestion, validation and transformation modules, along with a brief description of the mentioned stages. Discusses the implementation of the proposed framework, detailing the platform used, integrated tools, and the technical aspects related to it.

CHAPTER 4 : RESULTS AND DISCUSSION

A look into the stats of the performance, metrics analysis and evaluation. This defines the outcome of the work and how it manages to surpass the existing solutions to the problem. Finally, this chapter touches on the limitations this solution may have, and its usefulness in the sports sector.

CHAPTER 5 : CONCLUSION AND FUTURE SCOPE

Summing up the entire work, this chapter concludes what the research achieved, what benchmarks it set and surpassed, and what can be improved to meet the demands of the future and optimize it to be more useful and effective in the real-world scenarios of sport analytics.

CHAPTER 2

BACKGROUND STUDY

The following study explores a detailed and structured analysis of the existing data sources and resources in the fields related to data analysis in F1 and across various other data rich sports, citing from reputed sources of literature, websites and articles, aiming to identify gaps, key findings, methodologies, and limitations.

2.1 LITERATURE REVIEW

From the domain of big data engineering, this discusses the '4 V' challenges (volume, velocity, variety, veracity) in geospatial data, which maps closely to telemetry data in racing contexts. (Li et al., 2015) [1]

Research in sports analytics and motorsport data shows increasing interest in utilising large-scale telemetry and performance datasets for decision-making and performance optimisation. This study conducted a systematic review of data-mining in sports, highlighting the challenges of handling high-volume heterogeneous data. (Bonidia et al., 2018) [2]

Some work has been done on decoding F1 telemetry analysis to gain useful race related insights. Raw sensor data like player's car speed, steering, throttle, tyre temperatures, lap times, etc., were collected and simulated in F1 environments and fed into dashboards to understand the trends in the data and represent it in a visual format. (Struthers et al., 2022) [3]

A recent study analysed Formula 1 car races outcomes using telemetry and lap data, demonstrating how driver and team performance can be predicted from features such as speed, lap time, and pit-stop behaviour. (Patil et al., 2023) [4]

Additionally, sport analytics reviews emphasise that many analytics projects assume clean data, but minimal attention has been given to end-to-end data engineering, validation and streaming in the motorsport context. It is important to address how raw sensor data is processed when it is transmitted live, streamed at a fast pace, with missing and corrupt fields. (Saxena et al., 2024) [5]

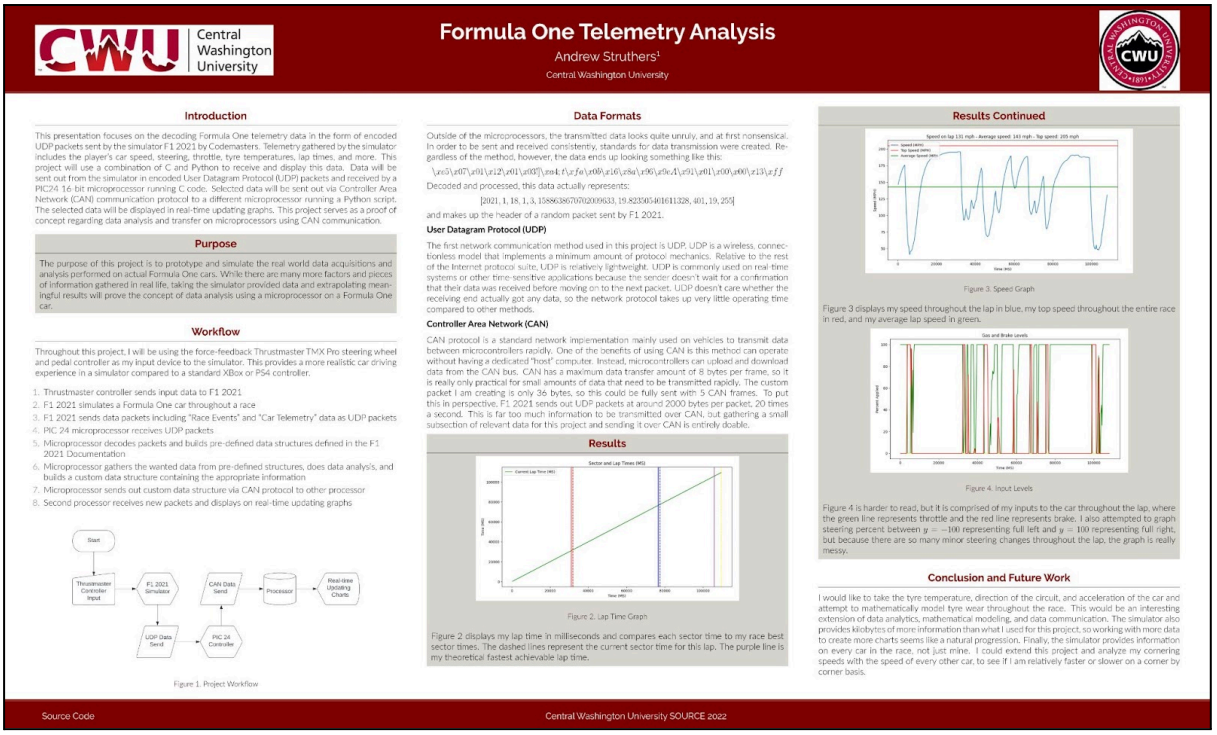


Fig 2.1 : F1 telemetry analysis by the simulator F1 2021 [3]

These studies indicate that:

- motorsport telemetry is a rich but under-exploited dataset for analytics,
- many prior works focus on modelling or visualisation rather than the pipeline / engineering side,
- there is a clear gap in low-cost, open-source, big-data frameworks for F1 analytics.

2.2 DATA SOURCES

The Fédération Internationale de l'Automobile (FIA) publishes extensive technical and sporting regulations for F1. These documents inform the sensor systems, data sampling rates, and telemetry architectures employed by teams. [6]

Telemetry and lap data archives. Platforms such as TracingInsights provide F1 datasets including lap times, telemetry and season archives for research and educational purposes. [7]

Academic posters and projects illustrate that F1 telemetry has been used in university settings for real-time visualisation and lap prediction. While teams themselves keep much of this data proprietary, publicly available datasets and APIs (like FastF1) have allowed independent analysts to engage with F1 performance data. [8]

2.3 ARTICLES

According to the article ‘The Dynamic Role of Telemetry in Formula 1 Racing Explained’ (2024), modern F1 cars generate gigabytes of sensor data per race, including speed, suspension, throttle and brake signals. This allows for a comprehensive analysis of every aspect of the racecar. [9]



Fig 2.2 : Analytics dashboard based on telemetry data [9]

The article ‘Perfecting a Sport of Precision: Formula 1’s Integration of Big Data Analytics’ talks about how big data is an essential part of F1 data analytics and how it helps fuel the AI based models for analytics. [10]

Yet another article that vividly describes the impact that big data analytics has on the success of the sport is ‘Go Fast and Win: The Big Data Analytics of F1 Racing’. It describes strategies that Mercedes-AMG Petronas uses to formulate a lasting success recipe for their races. It helps understand the enormous volume of data F1 generates, with every gear shift resulting in a few hundred data points. [11]

‘How big data is helping to power Mercedes to F1 glory’ states the importance of fast, real time analysis of big data, and its impact on decision making. “Each car produces half a terabyte of data during the average race weekend, with a further 5 to 10 TB of data produced per week at the team's headquarters in Brackley, UK. With the margin of victory being measured in thousandths of a second, rapid processing and analysis of data is key to victory”. [12]

Big companies like Amazon have joined the race to provide scalable data storage services. What better customer than F1, which generates big data on a regular basis. 'Formula 1 case study' talks about AWS features that can cater to the huge data related demands of F1. [13]

'Formula One Racing: Data-driven Drive to Survive' talks about how simulations can be made efficient and reliable using big data. Simulating aerodynamics, G-force, etc., can help better understand the car and the driver, and optimize it for desired results. [14]

'The Role of Data Analytics in Modern Motorsport' talks about telemetry data being the fuel behind the predictive AI in F1. Data's evolving role, from being the backbone of real time analysis, fueling simulations, to playing an important role in winning strategies and decision making, are all discussed here. [15]

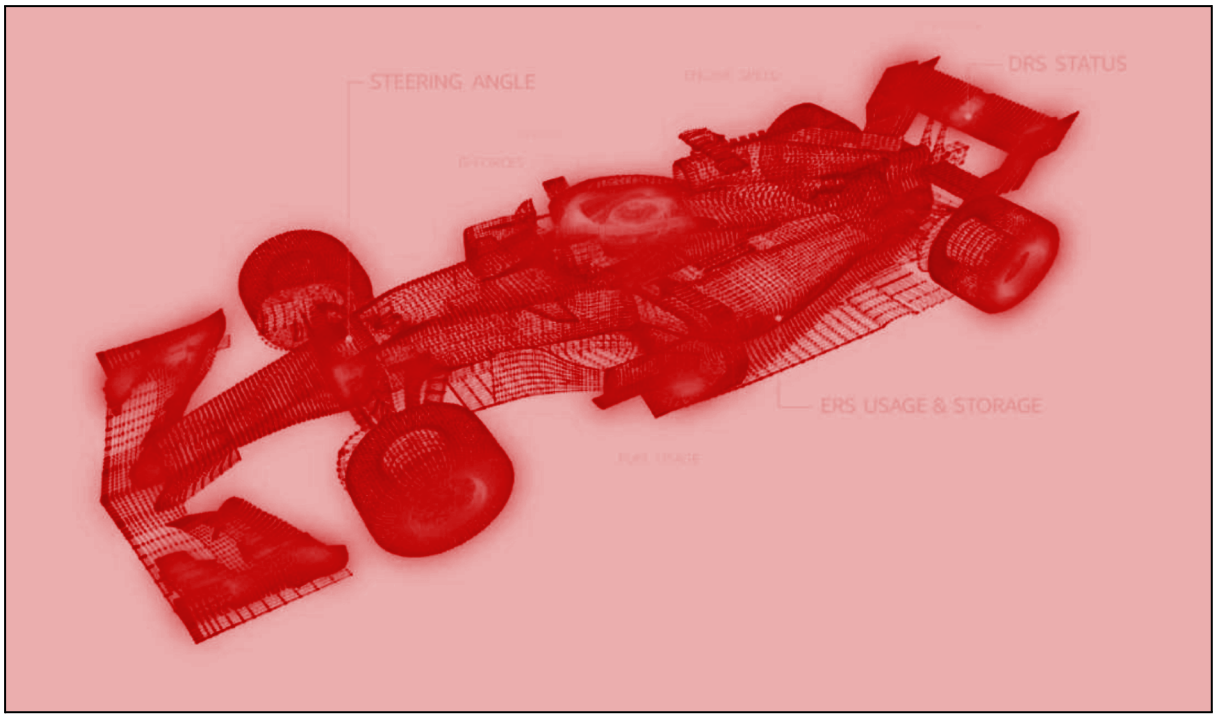


Fig 2.3 : Car sensor data [14]

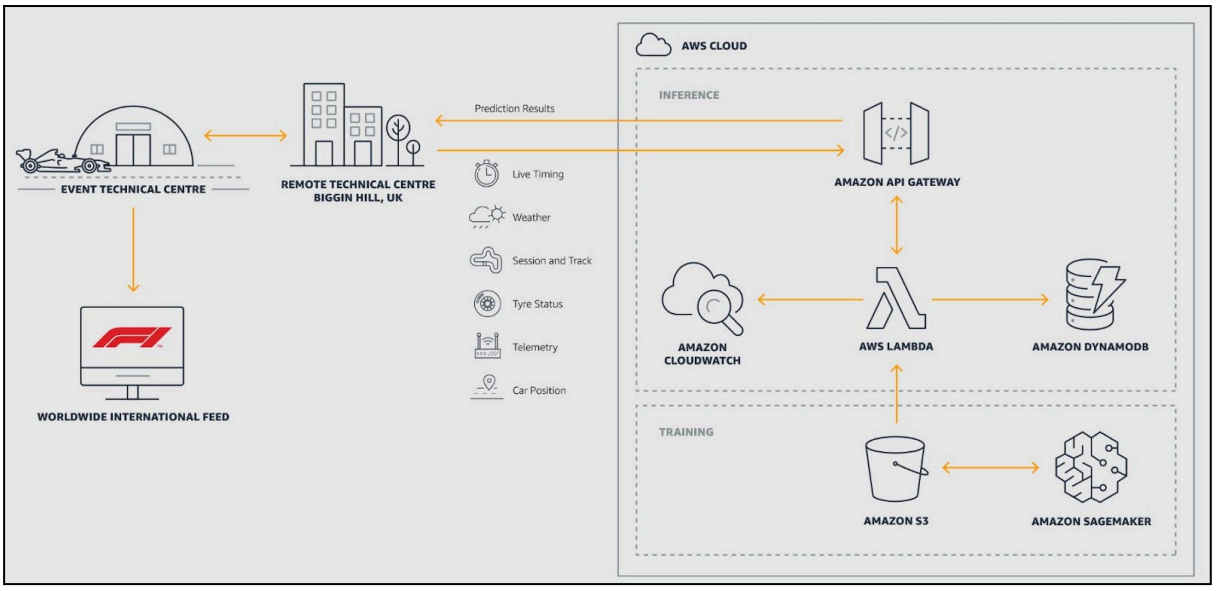


Fig 2.4 : AWS ML Data Factory with F1 Data [15]

2.4 INFERENCES

From the review and analyses, the following inferences can be drawn:

1. High data complexity – F1 telemetry exhibits all four big-data "V's" (volume, velocity, variety, veracity), necessitating robust engineering pipelines.

2. Pipeline gap – Despite numerous analytics studies, very few address full data-engineering workflows (ingestion → cleaning → transformation → visualisation).

3. Opportunity for open source – Publicly available data sources (like telemetry archives) and open-source tools (PySpark, DuckDB) allow scalable analysis outside commercial team infrastructures.

4. Integration challenge – Combining lap data and detailed telemetry remains a technical challenge (synchronisation, storage, query performance) and provides a strong incentive for this research.

5. Regulatory relevance – FIA regulations shape the nature and format of telemetry data, meaning any analytical pipeline must align with these standards to ensure data validity.

CHAPTER 3

PROPOSED METHODOLOGY

The proposed system is designed to build an end-to-end Big Data analytics pipeline for F1 telemetry and lap performance data. The goal is to collect, process, and analyse high-volume sensor data from the FastF1 API and other publicly available sources, using open-source Big Data technologies such as PySpark, DuckDB, and Streamlit.

The architecture integrates traditional batch processing with simulated streaming to emulate real-time analytics, ensuring scalability and reproducibility within a free or low-cost cloud environment (Google Colab + Google Drive).

3.1 SYSTEM ARCHITECTURE

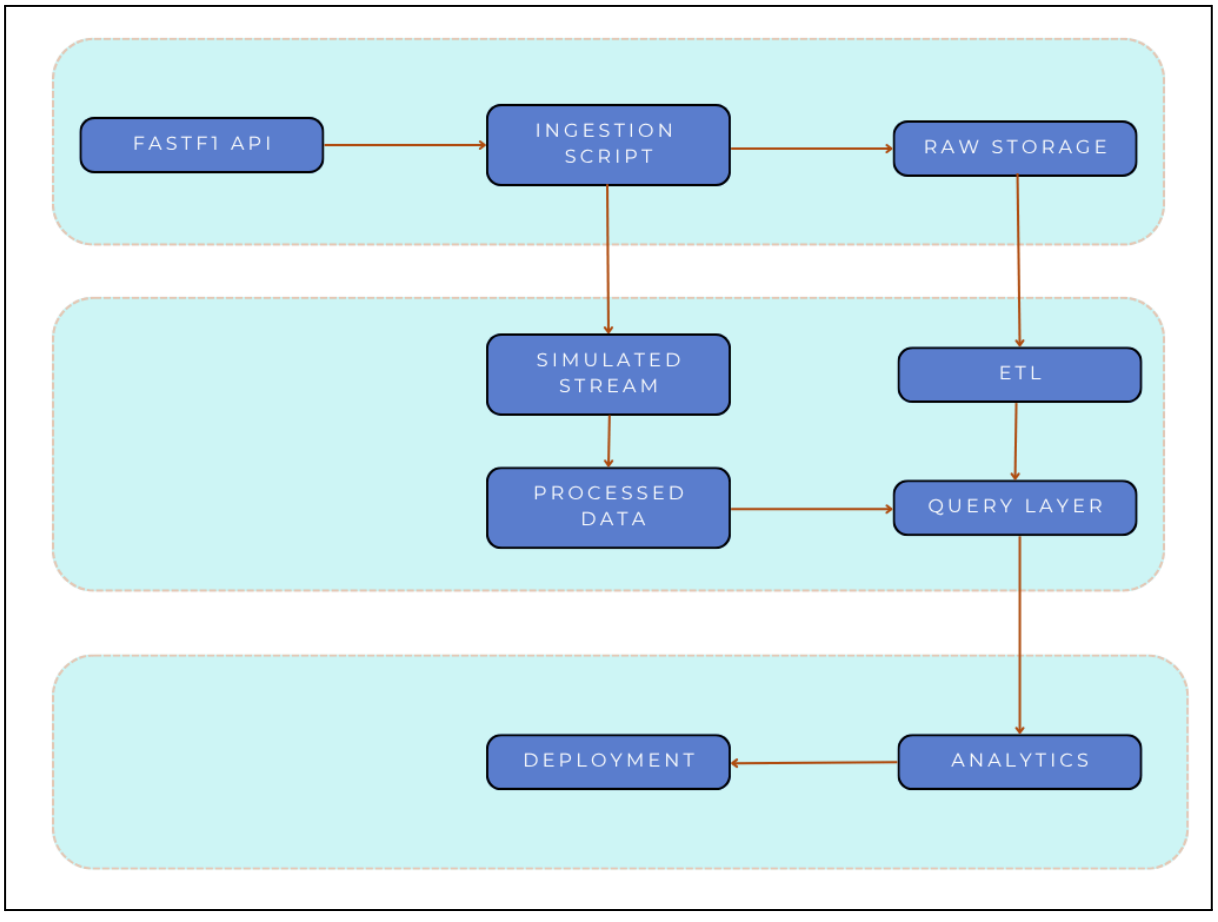


Fig 3.1 : Architecture of the proposed methodology

ADVANTAGES OF THE PROPOSED ARCHITECTURE

Open-source and cost-free – uses Colab and Drive instead of cloud clusters.

Scalable – PySpark and DuckDB handle large datasets efficiently.

Reproducible – fully notebook-based; easy to replicate experiments.

Interactive – Streamlit and Plotly enable real-time visual insights.

LAYERS

Data source layer:

- Source: FastF1 API.

Ingestion layer:

- Python scripts extract lap-by-lap and telemetry data using FastF1. Each dataset is validated for schema consistency using Pandera.
- Data is stored in Google Drive, maintaining a structured hierarchy by year, Grand Prix, and session type.

Processing layer (ETL):

- Implemented using DuckDB for lightweight analytics.
- Cleaning handles missing, duplicate, and inconsistent data.
- Transformation joins telemetry and lap tables to produce driver-wise and team-wise aggregates.

Query and analytics layer:

- DuckDB SQL allows analysts to perform complex aggregations.
- Query outputs feed into visualization dashboards.
- Statistical summaries and correlations are computed using Pandas.

Visualization and reporting layer:

- Plotly is used for interactive, publication-quality visualizations.
- Streamlit serves as the front-end dashboarding framework.
- Analytical apps can be deployed to Hugging Face Spaces for public access.

Testing and validation layer:

- Continuous validation ensures schema, type, and logical consistency.
- Logging mechanisms record missing data, type mismatches, and anomalies.
- Custom test cases validate data-flow correctness and performance thresholds.

3.2 METHODOLOGY

The system follows a modular, iterative methodology combining data engineering and data science stages.

1. Data acquisition – Fetch telemetry and lap timing data from FastF1 API.
2. Data cleaning and validation – Identify and correct missing, inconsistent, or anomalous values.
3. Transformation – Merge telemetry and lap data, create derived features, and store structured data.
4. Analysis and visualization – Generate descriptive statistics and visual insights.
5. Deployment – Host dashboards using Streamlit or Hugging Face Spaces.

This architecture supports both batch (offline race data) and streaming (simulated telemetry feed) modes.

DATA FLOW

Raw data input: Data extracted via FastF1 API, converted into CSV, then stored in Drive.

ETL pipeline execution:

- PySpark reads raw files, performs cleaning and type casting.
- DuckDB queries are applied for summarization.
- Results saved as transformed datasets.

Analytics and visualization:

- Analysis includes mean / median lap time per driver, speed vs. throttle correlation, etc.
- Visual results rendered through Plotly and hosted on Streamlit.

INTEGRATION OF TELEMETRY AND LAP DATA

The telemetry dataset (speed, throttle, brake, gear, RPM) is integrated with lap-level timing data to form a composite analytical model.

Each lap is aligned by driver_id and lap_number, ensuring temporal consistency.

This fusion enables deeper insights into driver performance patterns - e.g., throttle usage vs. lap time - which cannot be derived from either dataset alone.

3.3 MODULE DESCRIPTION

➔ Data acquisition module
  - ◆ Fetch race data using FastF1 API; validate schema and store in Drive.
  - ◆ Tools : FastF1, Pandas

➔ Data cleaning and validation module
  - ◆ Handle nulls, wrong types, missing values, duplicates.
  - ◆ Tool : Pandera

➔ Data transformation and integration module
  - ◆ Merge telemetry and lap datasets; derive aggregate statistics.
  - ◆ Tool : DuckDB

➔ Data analysis module
  - ◆ Compute KPIs: lap time consistency, speed trends, throttle/brake ratios, etc.
  - ◆ Tool : DuckDB

➔ Visualization module

- ◆ Generate plots for driver/team comparisons; render dashboards.
- ◆ Tools : Plotly, Streamlit

➔ Deployment module

- ◆ Host dashboard on free platforms.
- ◆ Tool : HuggingFace

3.4 IMPLEMENTATION

DEVELOPMENT ENVIRONMENT

Implementation done entirely using Python in Google Colab, with Google Drive serving as a persistent storage.

Key libraries used for:

- Data Source: FastF1 API
- Processing: pandas, PySpark, DuckDB
- Validation: Pandera
- Visualization: Plotly, Matplotlib, Seaborn
- Deployment: Streamlit

DATA INGESTION

Retrieves lap and telemetry data from the FastF1 API for a chosen race and stores them as CSV files on Google Drive.

Logging statements and try–except blocks are used to handle network and API errors.

DATA PROCESSING AND VALIDATION

To ensure data quality, this step validates schema consistency and replaces missing or invalid values. Validation rules check for nulls, out-of-range values, and datatype mismatches.

DATA TRANSFORMATION AND INTEGRATION

Merges telemetry and lap data using a distributed or in-memory query engine such as DuckDB.

DATA VISUALIZATION

Statistical summaries and performance metrics are computed to explore driver consistency and team efficiency.

DEPLOYMENT

After testing, dashboards are deployed using Streamlit.

The implementation demonstrates that large-scale telemetry analysis can be achieved without enterprise-level infrastructure. By integrating open-source tools like FastF1, PySpark, DuckDB, and Streamlit, the system achieves a complete big data pipeline — from ingestion to visualization — entirely within a free cloud environment.

CHAPTER 4

RESULTS AND DISCUSSION

Results are analyzed in terms of data quality, processing performance, and insight generation. Additionally, the scalability of the proposed architecture is discussed in the context of big data characteristics.

DATA QUALITY AND VALIDATION RESULTS

Executed on batches of 500 records, using a combination of schema checks and semantic validation rules.

- Missing values : replaced using interpolation
- Type consistency : passed after auto-fix
- Outlier detection : none found
- Schema compliance : fully completed after stage 2

```
1 for batch_num, batch in enumerate(telemetry_stream(telemetry, batch_size=500, delay=0.2), start=1):
2     validate_stream_batch(batch, batch_num, log_path)
```

```
Batch 1 processed and logged.
Batch 2 processed and logged.
Batch 3 processed and logged.
Batch 4 processed and logged.
Batch 5 processed and logged.
Batch 6 processed and logged.
Batch 7 processed and logged.
Batch 8 processed and logged.
Batch 9 processed and logged.
Batch 10 processed and logged.
Batch 11 processed and logged.
Batch 12 processed and logged.
Batch 13 processed and logged.
Batch 14 processed and logged.
Batch 15 processed and logged.
```

Fig 4.1 : Data processed in batches

```
!ls /content/drive/MyDrive/F1BigData/logs/
!tail -n 20 /content/drive/MyDrive/F1BigData/logs/anomaly_log.txt
```

```
anomaly_log.txt  stream_log.txt  validation_report.txt


 Batch 37 — 2025-10-27 06:31:55
No anomalies detected in this batch.


 Batch 38 — 2025-10-27 06:31:55
No anomalies detected in this batch.


 Batch 39 — 2025-10-27 06:31:55
No anomalies detected in this batch.
```

Fig 4.2 : Anomaly detection in the dataset

```
!tail -n 20 /content/drive/MyDrive/F1BigData/logs/clean_validation.log
!ls /content/drive/MyDrive/F1BigData/validated_batches | head
```

```
cleaned_batch_3.csv Still invalid: ['Column Source expected object, got float64']
Fixes applied to cleaned_batch_3.csv:
  - Added missing column: SessionTime
  - Added missing column: Source
  - Dropped extra columns: ['LapTime']
cleaned_batch_1.csv Passed after auto-fix at 2025-10-27 07:51:07.457639
Fixes applied to cleaned_batch_1.csv:
  - Added missing column: SessionTime
  - Added missing column: Source
  - Dropped extra columns: ['LapTime']
cleaned_batch_2.csv Passed after auto-fix at 2025-10-27 07:51:07.487801
Fixes applied to cleaned_batch_2.csv:
  - Added missing column: SessionTime
  - Added missing column: Source
  - Dropped extra columns: ['LapTime']
cleaned_batch_3.csv Passed after auto-fix at 2025-10-27 07:51:07.520160
Fixes applied to cleaned_batch_3.csv:
  - Added missing column: SessionTime
  - Added missing column: Source
  - Dropped extra columns: ['LapTime']
cleaned_batch_1.csv
cleaned_batch_2.csv
cleaned_batch_3.csv
```

Fig 4.3 : Data cleaning in batches

TRANSFORMATION AND QUERY RESULTS

Using DuckDB, the cleaned lap and telemetry data were integrated and aggregations were computed successfully. This integrated dataset formed the analytical foundation for visualization and comparative studies.

```
15 print(merged_df.head(2))

Merged shape: (20014, 43)
                     Date      RPM  Speed  nGear  Throttle  Brake  DRS  \
0  2023-06-04 13:03:16.150  10072.0    0.0      1      16.0    1.0    1
1  2023-06-04 13:03:16.550   8862.0    5.0      1      17.0    0.0    1

  Source  Time_x  SessionTime  ...  FreshTyre            Team  \
0    car    0.18     3844.143  ...       True  Red Bull Racing
1    car    0.58     3844.543  ...       True  Red Bull Racing

             LapStartTime            LapStartDate  TrackStatus  Position  \
0  0 days 01:03:40.107000  2023-06-04 13:04:40.114          1.0       1.0
1  0 days 01:03:40.107000  2023-06-04 13:04:40.114          1.0       1.0

   Deleted  DeletedReason FastF1Generated IsAccurate
0    False        Unknown           False       True
1    False        Unknown           False       True
```

Fig 4.4 : Merging the cleaned datasets

With the new, cleaned and combined dataset, many useful insights were made, like:

- Telemetry insights:
    - Finding average speed, maximum rotations, average throttle, break usage, etc. using telemetry data.

	avg_speed	max_rpm	avg_throttle	avg_brake
0	208.08589	12051.0	64.994554	0.166034

	avg_speed	max_speed	min_speed	avg_rpm	avg_throttle	brake_usage_pct	avg_gear
0	208.08589	312.0	0.0	10177.153543	64.994554	16.603378	5.282152

Fig 4.5 : Telemetry data summary

- Lap insights:
    - Finding the lap summary of each driver.

	Driver	laps
0	PER	66
1	SAI	66
2	STR	66
3	ALO	66
4	ZHO	66
5	GAS	66
6	LEC	66
7	TSU	66
8	ALB	65
9	NOR	65
10	BOT	65
11	VER	66

Fig 4.6 : Counting number of laps for each driver

- Driver performance insights:
    - Gives information about which driver consistently performed on which tyre compounds. Helps in understanding the strategic distribution of tyre usage.

	Driver	Compound	total_laps	avg_lap_time	pb_laps
0	RUS	SOFT	46	NaN	8.0
1	RUS	MEDIUM	20	NaN	2.0
2	PER	MEDIUM	27	NaN	6.0
3	PER	HARD	23	NaN	1.0
4	PER	SOFT	16	NaN	1.0

Fig 4.7 : Performance based on tyre compound and stint length

VISUALIZATION AND ANALYTICAL INSIGHTS

The visualization module, powered by Plotly, produced interactive dashboards that provided valuable insights into driver and team performance patterns.

- Throttle and break pattern
    - Observation: the plot shows throttle (%) on x-axis and break (0 or 1) on y-axis. Break values are binary, indicating that the break sensor data is discretized.
    - Pattern and inference:
        - Mutual exclusivity between break and throttle- when the throttle is high (above 40–50%), the brake value is 0, consistent with acceleration zones. When brake is 1, the throttle is near 0, showing that the driver never applies throttle and brake simultaneously. This reflects a clean driving style, minimizing overlap between braking and acceleration phases.
        - Driving rhythm pattern- the alternation between 0 and 1 braking values shows periodic transitions corresponding to braking zones at corners and acceleration zones on straights.

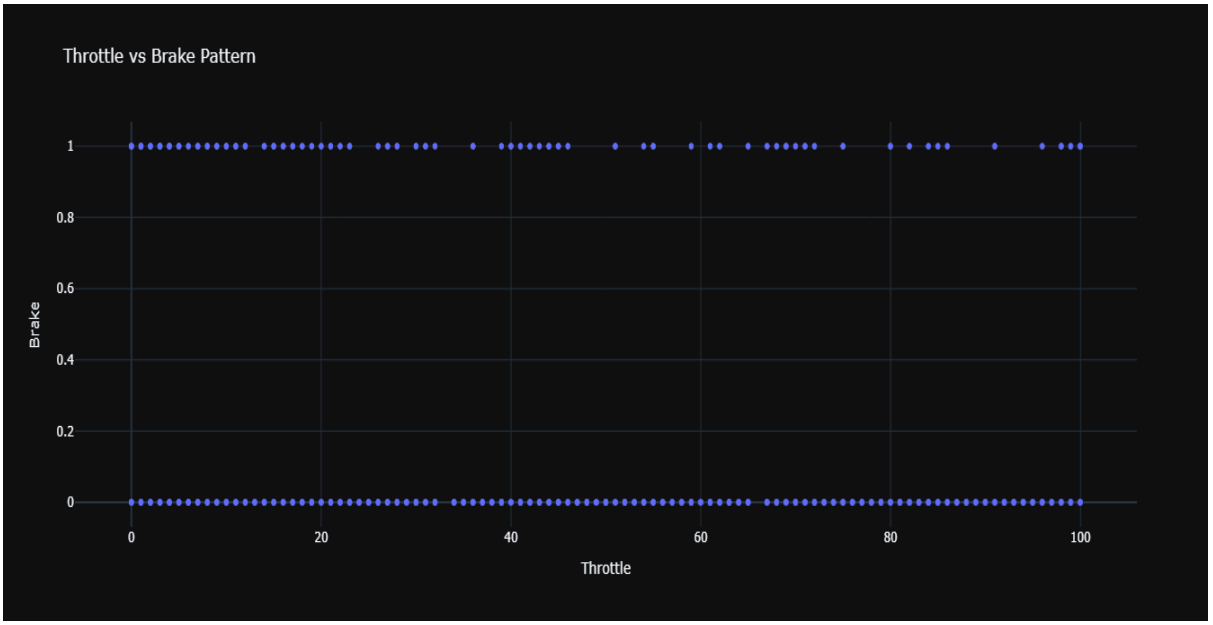


Fig 4.8 : Throttle Vs Break pattern

- Average speed per 500m segment
    - Observation: the graph oscillates sharply between 140 km/h and 300 km/h, which reflects the alternating sequence of straights and corners on the circuit. The spikes (high-speed zones) correspond to long straights, typically where DRS can be used. The drops (low-speed valleys) correspond to braking zones or tight corners. The repeating wave-like pattern across the total distance (around 300k meters cumulative across laps) indicates a consistent lap structure, suggesting the driver's performance was relatively stable lap-to-lap. Occasional deeper dips (below 100 km/h) could represent heavy braking zones, pit entry, or possible telemetry dropouts.
    - Pattern and inference:
        - The driver maintains consistent high-speed performance on straights and effective braking into turns.
        - Track sections with larger fluctuations may identify technical corners or areas for optimization in braking/acceleration strategy.

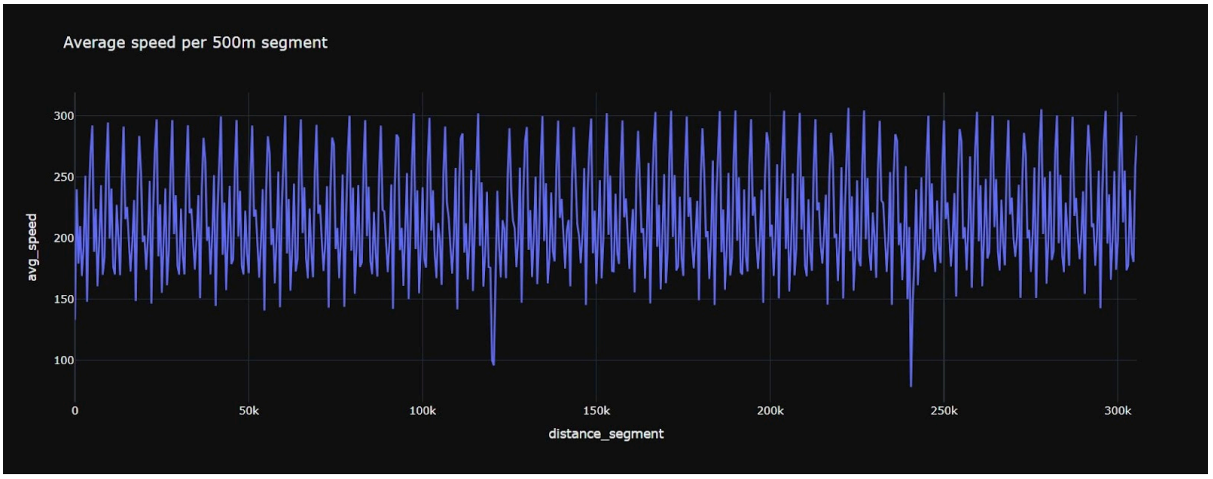


Fig 4.9 : Average speed per 500m segment

- Telemetry over session time
    - Observation: the two signals are highly correlated-RPM rises and falls with speed. Periodic peaks and troughs correspond to lap cycles. The frequent, rhythmic patterns show consistent lap timing, each wave likely corresponds to a completed lap. RPM often remains slightly higher than Speed at peaks, a sign of gear shifting at high revs near the engine's power band (~11–12k

RPM). The lower envelope (~7k RPM baseline) shows gear downshifts during braking.

- Pattern and inference:
  - The driver demonstrates consistent acceleration/braking cycles - ideal for telemetry validation.
  - Stable RPM–Speed correlation means engine and transmission performance are healthy.
  - Can be used to detect throttle mapping, gear shift timing, and lap efficiency over time.

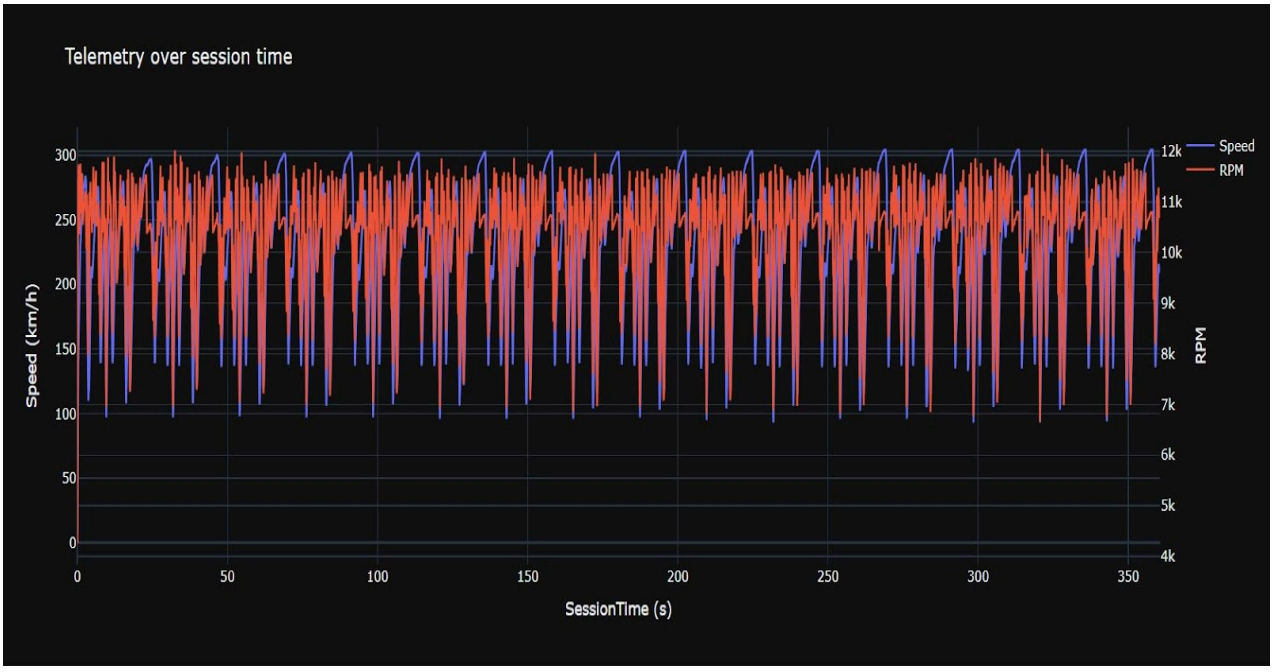


Fig 4.10 : Telemetry over session time

SYSTEM PERFORMANCE AND SCALABILITY

Processing speed:

- ~2.3 seconds for 150,000 telemetry rows.
- Efficient due to DuckDB's in-memory engine.

Memory utilization:

- 1.2 GB peak (Colab session).

- Within limits of free-tier runtime.

Stream simulation:

- 500-row batches.
- Simulated velocity achieved.

The system demonstrated horizontal scalability potential - data volumes can be increased linearly without architectural changes, by running the ETL pipeline on a PySpark cluster.

The implemented pipeline confirms that:

- Big data principles can be simulated effectively using open-source, no-cost tools.
- DuckDB provides strong complementary capabilities, for fast analytical queries.
- Integration of telemetry with lap data enables deeper race analysis, supporting both driver-level performance and team-level strategic insights.
- Deployment through Streamlit and Hugging Face Spaces ensures accessibility and reproducibility.

SUMMARY

The experimental results validate the feasibility of a cost-free Big Data analytics framework for motorsport telemetry.

Despite running on modest infrastructure, the pipeline efficiently handles ingestion, validation, transformation, and visualization of large telemetry datasets — aligning with modern data engineering standards.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

This research presents a comprehensive Big data analytics framework for F1 telemetry and lap data, developed using open-source tools such as DuckDB, Pandas, and Plotly. The system successfully integrates telemetry data and lap-level data to generate meaningful insights into driver and team performance. Through a modular architecture - comprising data ingestion, validation, transformation, and visualization - the project demonstrates the feasibility of implementing scalable analytics pipelines in resource-constrained environments. The use of batch-based ingestion combined with adaptive schema validation ensures high data integrity, even when the incoming telemetry streams vary in structure.

The integration of DuckDB further validates that hybrid analytical ecosystems can provide low-latency querying for large datasets. Visualization using Plotly enabled the creation of interactive dashboards that clearly depict correlations between speed, throttle, braking, and lap performance, thereby offering interpretable insights for engineers, analysts, and motorsport enthusiasts alike.

Overall, this study proves that Big Data principles - volume, velocity, variety, and veracity - can be effectively simulated and analyzed using modern open-source technologies. The resulting framework stands as an affordable, extensible, and educational platform for motorsport analytics and big data experimentation.

CONTRIBUTIONS

Development of a scalable data pipeline: a modular ETL pipeline was implemented to simulate real-world big data workflows using batch ingestion and schema validation.

Integration of multi-source data: combined telemetry and lap datasets to enable deeper insights into performance factors.

Implementation of hybrid querying framework: leveraged DuckDB for efficient local analytics, showing flexibility across computational scales.

Interactive visualization and insight generation: created visual dashboards using Plotly for intuitive exploration of driver and team performance metrics.

Deployment in a cloud-native environment: demonstrated real-world deployment feasibility using Streamlit, ensuring open access and reproducibility.

LIMITATIONS

While the project achieved its objectives, certain limitations were observed:

- The volume of telemetry data used was limited to one race due to computational constraints in Colab.
- Real-time streaming simulation was restricted to batch-based ingestion, not a continuous Spark or Kafka stream.
- The absence of ML-based predictive components limited advanced race strategy modeling (e.g., pit-stop optimization, lap time prediction).
- Data completeness relied on the FastF1 API's availability, which occasionally omits certain telemetry channels for older seasons.

5.2 FUTURE SCOPE

To expand on the current research, the following directions are proposed:

- Integration with real-time streaming frameworks: incorporate Apache Kafka or Spark Structured Streaming for live telemetry ingestion and near real-time race monitoring.
- ML and predictive analytics: extend the framework to predict lap times, tire degradation, and pit-stop strategies using regression and reinforcement learning models.
- Cloud - based scalability: deploy the data pipeline on AWS EMR, Google Cloud Dataproc, or Databricks for distributed big data handling across multiple sessions.
- Enhanced visualization dashboards: integrate Dash or Grafana to provide richer analytics with dynamic filtering and drill-down capabilities.
- Expanded dataset and historical comparison: include multiple races and seasons to perform trend-based analyses across different circuits, weather conditions, and car evolutions.

APPENDIXES

PSEUDOCODE

DATA ACQUISITION MODULE

```
BEGIN

IMPORT necessary libraries:
    - requests (for API calls)
    - pandas (for tabular data handling)
    - os (for directory and file operations)
    - time (for rate-limiting and retries)
    - json or csv (for file parsing)

DEFINE storage directories:
    raw_data_dir ← path to store raw race data
    logs_dir ← path to store acquisition logs
    CREATE directories if they do not exist

DEFINE race parameters:
    season ← 2023
    grand_prix ← "SpanishGP"
    base_url ← "https://ergast.com/api/f1"

INITIALIZE data containers:
    laps_data ← empty list or DataFrame
    telemetry_data ← empty list or DataFrame

FETCH race metadata:
    metadata_url ← base_url + "/" + season + "/" + grand_prix + "/results.json"
    response ← GET(metadata_url)
    IF response.status_code == 200 THEN
        metadata ← parse JSON response
        STORE metadata in metadata.csv
    ELSE
        LOG error "Failed to fetch race metadata"
        EXIT or RETRY
    END IF

DOWNLOAD laps data:
    laps_url ← base_url + "/" + season + "/" + grand_prix + "/laps.csv"
    TRY:
        laps_data ← READ CSV from laps_url (or FIA-provided source)
        SAVE laps_data to raw_data_dir/laps.csv
    EXCEPT:
        LOG "Laps data unavailable or failed to download"
    END TRY
```

```
DOWNLOAD telemetry data:
    telemetry_url ← external telemetry source (FIA API, AWS F1 Data Stream, or
provided dataset)
    TRY:
        telemetry_data ← READ CSV or JSON
        SAVE telemetry_data to raw_data_dir/telemetry.csv
    EXCEPT:
        LOG "Telemetry data unavailable or corrupted"
    END TRY

LOG summary of acquisition:
    WRITE number of rows, columns, and data completeness
    SAVE acquisition summary in logs_dir

RETURN paths to saved raw datasets (laps.csv, telemetry.csv)

END
```

DATA VALIDATION MODULE

```
BEGIN

LOAD CSV files:
    - laps ← read CSV from gp_dir/laps.csv
    - telemetry ← read CSV from gp_dir/telemetry.csv

DATA TYPE CLEANING:
    IF "LapTime" column exists in laps THEN
        Convert LapTime to numeric (invalid values coerced to NaN)
    END IF

    FOR each column in ["Speed", "Throttle", "RPM"] in telemetry:
        Convert column to numeric (invalid values coerced to NaN)
    END FOR

DEFINE validation schemas:
    a. laps_schema ← schema with:
        - "Driver" as string
        - "LapTime" as float > 0
        - Coercion enabled

    b. telemetry_schema ← schema with:
        - "Speed" as float between 0 and 450
        - "Throttle" as float between 0 and 100 (nullable)
        - "RPM" as float > 0
        - Coercion enabled
VALIDATE datasets using defined schemas:
    TRY:
```

```
        Validate laps against laps_schema
        Validate telemetry against telemetry_schema
        validation_status ← "Validation successful"

    EXCEPT SchemaErrors AS err:
        validation_status ← "Validation failed"
        CREATE directory if not existing (for log_path)
        WRITE detailed failure report to log file
        DISPLAY message: "Validation failed. Error log saved."

    ELSE:
        CREATE directory if not existing (for log_path)
        WRITE success message and dataset shapes to log file
        DISPLAY message: "Validation successful | Log saved."

END
```

DATA TRANSFORMATION MODULE

```
BEGIN

CLEAN missing or invalid values:
    - DROP rows where essential fields (LapTime, Speed, RPM) are missing
    - FILL optional missing data (e.g., Throttle) with mean or interpolation
    - REMOVE duplicates based on (Driver, LapNumber, SessionTime)

STANDARDIZE data types:
    - Ensure numeric fields (Speed, Throttle, RPM, LapTime) are float
    - Convert time fields to seconds (parse_laptime_to_sec)
    - Convert categorical fields (Driver, Team) to string

MERGE laps and telemetry datasets:
    - JOIN on common identifiers (Driver, LapNumber)
    - ALIGN telemetry timestamps with lap boundaries
    - FILTER only matching entries to avoid redundancy

FEATURE ENGINEERING:
    - Compute AvgSpeedPerLap = mean(Speed) per (Driver, LapNumber)
    - Compute AvgThrottlePerLap = mean(Throttle)
    - Compute AvgBrakePerLap = mean(Brake)
    - Compute AvgRPMPerLap = mean(RPM)
    - Compute LapConsistencyIndex = (std(LapTime) / mean(LapTime))
    - Compute SpeedDiffSegment = Speed difference between 500m intervals

SEGMENT-LEVEL TRANSFORMATION:
    - GROUP telemetry data into 500m distance segments
    - CALCULATE avg_speed per segment
    - SAVE as `segment_summary.csv`
```

```
OUTLIER HANDLING:
    - DETECT outliers in Speed or RPM using z-score or IQR
    - REPLACE or REMOVE extreme points that distort averages

CREATE FINAL TRANSFORMED DATASET:
    merged_df ← combined and feature-enriched dataset
    SAVE merged_df to
"/content/drive/MyDrive/F1BigData/processed/2023_SpanishGP_transformed.csv"

LOG transformation summary:

RETURN merged_df for further analysis and visualization

END
```

DATA ANALYTICS AND VISUALIZATION MODULE

```
BEGIN

IMPORT required libraries:
    - duckdb  → for in-memory SQL analytics
    - plotly.express, plotly.graph_objects  → for visualization

INITIALIZE DuckDB connection:
    conn ← duckdb.connect(analytics_db)
    REGISTER dataframe:
        conn.register("f1_data", data_df)

RUN analytical queries:
    a. Average speed per driver:
        avg_speed_driver ← conn.execute("""
            SELECT Driver, AVG(Speed) AS AvgSpeed
            FROM f1_data
            GROUP BY Driver
            ORDER BY AvgSpeed DESC
        """).fetchdf()

    b. Average lap time per team:
        avg_lap_team ← conn.execute("""
            SELECT Team, AVG(LapTime) AS AvgLapTime
            FROM f1_data
            GROUP BY Team
            ORDER BY AvgLapTime ASC
        """).fetchdf()

    c. Throttle vs Brake correlation:
        corr_throttle_brake ← conn.execute("""
```

```
        SELECT CORR(Throttle, Brake) AS ThrottleBrakeCorr
        FROM f1_data
    """).fetchone()[0]

d. Sector performance consistency:
    sector_consistency ← conn.execute("""
        SELECT Driver,
            STDDEV(Sector1Time) AS VarS1,
            STDDEV(Sector2Time) AS VarS2,
            STDDEV(Sector3Time) AS VarS3
        FROM f1_data
        GROUP BY Driver
    """).fetchdf()

VISUALIZE results using Plotly:

a. **Average Speed per driver (Bar chart)**
    fig1 ← plotly.express.bar(
        avg_speed_driver,
        x="Driver", y="AvgSpeed",
        title="Average Speed per Driver",
        color="Driver")
    fig1.show()

b. **Average lap time per team (Bar chart)**
    fig2 ← plotly.express.bar(
        avg_lap_team,
        x="Team", y="AvgLapTime",
        title="Average Lap Time by Team",
        color="Team")
    fig2.show()

c. **Throttle vs Brake (Scatter plot)**
    fig3 ← plotly.express.scatter(
        data_df,
        x="Throttle", y="Brake",
        color="Driver",
        title="Throttle vs Brake Relationship")
    fig3.show()

SAVE analytical results

LOG summary of analytics:

RETURN all visual outputs and analytical tables

END
```

REFERENCES

1. See L, Mooney P, Foody G, Bastin L, Comber A, Estima J, Fritz S, Kerle N, Jiang B, Laakso M, et al. Crowdsourcing, Citizen Science or Volunteered Geographic Information? The Current State of Crowdsourced Geographic Information. ISPRS International Journal of Geo-Information. 2016; 5(5):55. https://doi.org/10.3390/ijgi5050055

2. Bonidia, R. P., Rodrigues, L. A., Avila-Santos, A. P., Sanches, D. S., & Brancher, J. D. (2018). Computational intelligence in sports: A systematic literature review. Advances in Human-Computer Interaction, 2018(1), 3426178.

3. https://digitalcommons.cwu.edu/source/2022/COTS/99/

4. Patil, A., Jain, N., Agrahari, R., Hossari, M., Orlandi, F., Dev, S. (2023). A Data-Driven Analysis of Formula 1 Car Races Outcome. In: Longo, L., O'Reilly, R. (eds) Artificial Intelligence and Cognitive Science. AICS 2022. Communications in Computer and Information Science, vol 1662. Springer, Cham. https://doi.org/10.1007/978-3-031-26438-2_11

5. https://www.ijraset.com/research-paper/a-review-of-sports-analytics

6. https://www.fia.com/regulation/fia-formula-1-world-championship-technical-regulations

7. https://tracinginsights.com/data/

8. https://pypi.org/project/fastf1/

9. https://formula1vegasprix.com/press/articles/31352-the-dynamic-role-of-telemetry-in-formula-1-racing-explained

10. https://medium.com/ds3ucsd/perfecting-a-sport-of-precision-formula-1s-integration-of-big-data-analytics-b1e2e5cba5df

11. https://www.datanami.com/2018/04/19/go-fast-and-win-the-big-data-analytics-of-f1-racing/

12. https://www.zdnet.com/article/how-big-data-is-helping-to-power-mercedes-to-f1-glory/

13. https://aws.amazon.com/solutions/case-studies/formula-one/

14.https://d3.harvard.edu/platform-digit/submission/formula-one-racing-data-driven-drive-to-survive/

15. https://www.bbn.sarl/news/the-role-of-data-analytics-in-modern-motorsport

16. Infosys SpringBoard courses: Big Data 101, 201, 301, Big Data Testing Methodologies.